

Adaptive Trust-Oriented Runtime Security for AI-Generated Shell Scripts

Denys Yakymov

ORCID: 0009-0005-2398-8976 · yakden@gmail.com

Abstract—Large language models (LLMs) are increasingly deployed to generate shell scripts autonomously, creating an emerging attack surface in which syntactically valid yet semantically hazardous code may reach production pipelines without adequate scrutiny. This paper introduces the Adaptive Context-Aware Trust Execution (ACTE) framework, a runtime security system that assigns continuous risk scores to shell scripts through a logistic trust model combining semantic parsing, execution-context analysis, threat-intelligence signature matching, and online stochastic-gradient-descent (SGD) feedback learning. ACTE maps each script to one of four discrete trust levels—TRUSTED, MONITOR, RESTRICT, or DENY—and synthesizes corresponding enforcement artifacts (seccomp profiles, Open Policy Agent Rego policies, namespace/cgroup constraints). We evaluate ACTE on a reproducible dataset of 420 labeled scripts spanning five categories and three research questions. The full model achieves $F1 = 0.915$, $\text{Precision} = 0.985$, $\text{Recall} = 0.855$, and $\text{ROC-AUC} = 0.976$ at a mean analysis latency of 0.51 ms/script (approximately 1,963 scripts/s), while reducing the false-positive rate by 93.8% relative to ShellCheck v0.9.0. Ablation experiments confirm that threat-intelligence features contribute most to discriminative power ($\Delta F1 = -0.105$ when removed), whereas feedback learning is the principal guardian against false positives (FPR rises from 0.011 to 0.239 when removed). Beyond the single held-out split, five-fold cross-validation confirms the result is stable (stratified $F1 = 0.904 \pm 0.033$), and a stricter leave-template-out cross-validation—in which no generating template is shared between the training and test folds—still attains $F1 = 0.815 \pm 0.191$, directly rebutting the "template-memorization" objection to synthetic corpora. Bootstrap 95% confidence intervals ($F1: [0.861, 0.959]$) and an exact McNemar test ($p < 10^{-20}$ versus ShellCheck) establish the significance of these differences. Finally, on an independent hand-authored holdout of 41 real, publicly-documented scripts—used for testing only—a model trained solely on the synthetic corpus generalizes to $F1 = 0.950$ at $\text{Precision} = 1.000$ and $\text{FPR} = 0.000$. We report all results faithfully, including the synthetic nature of the training corpus and the prototype-sketch status of live enforcement.

Index Terms—runtime security, shell-script analysis, AI-generated code, trust evaluation, logistic risk model, online learning, seccomp, Open Policy Agent, zero-trust execution.

I. INTRODUCTION

The proliferation of large language models capable of generating executable code has fundamentally altered the software supply-chain landscape. Tools such as GitHub Copilot, ChatGPT, and their successors are now routinely employed to produce shell scripts for infrastructure management, deployment automation, and one-off administrative tasks. While this accelerates development velocity, it introduces a category of risk that conventional static analysis tools were not designed to address: the AI-generated shell script as an attack surface.

Shell scripts are privileged, imperative artifacts. A single malicious or misconfigured script can exfiltrate credentials, establish persistence, destroy filesystems, or exfiltrate data over covert channels. The key threat model concern is not that LLMs are inherently malicious, but rather that (i) prompt injection and adversarial prompting can induce dangerous outputs, (ii) model hallucination can produce

syntactically valid yet semantically harmful constructs, and (iii) the opacity of LLM generation means that standard code-review heuristics fail to capture the full behavioral envelope of the generated code.

Existing defenses are inadequate for this threat model. Static linters such as ShellCheck [1] detect syntactic errors and style violations but are agnostic to intent and behavioral context. Mandatory access-control frameworks such as SELinux [2] and AppArmor [3] operate at the process level and require pre-authored policy that cannot adapt to unseen code at runtime. Behavioral monitors such as Falco [4] and eBPF-based auditing systems [5] observe execution but cannot preempt it. None of these approaches integrates a continuous risk score derived from the script's content, execution context, and historical feedback into a unified adaptive trust decision made *before* the script runs.

We present the **Adaptive Context-Aware Trust Execution (ACTE)** framework to address this gap. ACTE's central

contribution is a formal logistic risk function $R(s)$ that aggregates features from three orthogonal evidence dimensions—semantic structure, execution context, and threat-intelligence signatures—into a continuous risk score in $(0, 1)$, which is then inverted to a trust score $T(s) = 1 - R(s)$ and mapped to a discrete enforcement level. Critically, the weight vector governing this scoring is updated online via SGD whenever labeled feedback is available, making ACTE adaptive to deployment-specific script distributions without requiring model retraining from scratch.

This paper makes the following contributions. First, we formalize the problem of adaptive trust-oriented runtime security for AI-generated shell scripts, including a precise threat model and effectiveness criteria (Section II). Second, we present the ACTE formal mathematical model and its distinguishing properties relative to prior art (Sections III–IV). Third, we describe the eight-module system architecture and its Python prototype implementation (Sections V–VI). Fourth, we report reproducible experimental results addressing detection accuracy, computational cost, and false-positive reduction (Sections VII–IX). Finally, we situate ACTE within the broader landscape of runtime security research and outline a planned program of follow-on work (Sections X–XIII).

II. PROBLEM FORMALIZATION

A. Object of Study

Let $\mathcal{S}$ denote the universe of POSIX-compatible shell scripts. Each script $s \in \mathcal{S}$ is a finite sequence of characters that may be submitted to a shell interpreter for execution on a host system with resources $\mathcal{R}$ (filesystems, network interfaces, device nodes, process table). We are interested in the subset $\mathcal{S}_{\text{AI}} \subseteq \mathcal{S}$ of scripts produced, in whole or in part, by generative language models, though the framework is applicable to any shell script regardless of provenance.

B. Threat Model

We consider an adversary $\mathcal{A}$ who controls the prompt submitted to an LLM-based code-generation service and whose goal is to cause the generated script to perform one or more of the following: (i) unauthorized data exfiltration, (ii) persistence establishment, (iii) privilege escalation, (iv) irreversible data destruction, or (v) lateral movement. We assume the adversary can craft obfuscated scripts (e.g., base64-encoded payloads, eval-based indirection, character-substitution ciphers) to evade pattern-based defenses. We do not model adversaries with physical hardware access or

kernel-level root kits; our threat boundary ends at the system-call interface.

C. Formal Definitions

Let $\mathcal{C}$ be the set of distinct commands (executables) that may appear in scripts in $\mathcal{S}$. Let $\mathcal{X} : \mathcal{S} \rightarrow [0, 1]^d$ be a feature-extraction function mapping any script to a d -dimensional real-valued feature vector. Define a *risk function* $R : \mathcal{S} \rightarrow (0, 1)$ and a *trust function* $T : \mathcal{S} \rightarrow (0, 1)$ such that $T(s) = 1 - R(s)$ for all s . The *trust level* of a script is a discrete label $\ell(s) \in \{\text{TRUSTED}, \text{MONITOR}, \text{RESTRICT}, \text{DENY}\}$ obtained by thresholding $R(s)$.

D. Research Questions

We structure the experimental evaluation around four research questions (RQs):

RQ1 (Detection Accuracy): What classification performance does ACTE achieve on held-out labeled scripts; how stable is that performance under cross-validation; and how does each component contribute?

RQ2 (Computational Cost): Is ACTE's per-script analysis latency compatible with inline deployment on interactive or CI/CD pipelines?

RQ3 (False-Positive Reduction): By how much does ACTE reduce the false-positive rate relative to a ShellCheck v0.9.0 baseline, and is the difference statistically significant?

RQ4 (Generalization to Real Scripts): Does a model trained solely on the synthetic corpus generalize to independent, real, publicly-documented scripts it has never seen?

III. THE ACTE MODEL AND NOVELTY

ACTE's distinguishing characteristics relative to existing work are as follows. First, it operates *before* execution, producing a trust decision based on static content analysis rather than observable runtime behavior—thus it can preempt dangerous scripts rather than merely detect them post-hoc. Second, unlike rule-only systems (SELinux, AppArmor, seccomp), ACTE derives its risk signal from the script's content and synthesizes a corresponding policy automatically; human policy authoring is not required per script. Third, unlike pure static linters, ACTE incorporates a probabilistic risk model with online learning, so its decision boundary adapts to evolving script distributions. Fourth, ACTE provides a continuous risk score rather than a binary flag, enabling graded enforcement proportionate to assessed risk.

The four trust levels map to concrete enforcement postures: TRUSTED scripts execute without restriction; MONITOR scripts execute with full audit logging; RESTRICT scripts execute in a sandboxed namespace with network egress blocked, read-only root filesystem, and capped CPU/memory/PID resources; DENY scripts are blocked entirely with no execution permitted.

The online feedback loop closes the adaptation loop: when a MONITOR-level script is later confirmed safe or dangerous by an operator, a single SGD step updates the weight vector in sub-millisecond time, moving the decision boundary for future similar scripts without requiring batch retraining.

IV. FORMAL MATHEMATICAL MODEL

A. Feature Space and Sets

Let $\mathcal{C}$ be the command set extracted by the SemanticParser, and let $\mathcal{G} = \{\text{semantic}, \text{context}, \text{threat}\}$ be the three feature groups. The full feature vector is $\mathbf{x}(s) \in [0, 1]^{13}$, with 5 semantic features, 5 context features, and 3 threat-intelligence aggregates (see Section V for definitions).

B. Risk Function

The logit (linear combination) is

$$z(s) = b + \sum_{i=1}^{13} w_i x_i(s),$$

where b is a scalar bias and $\mathbf{w} \in \mathbb{R}^{13}$ is the weight vector.

The risk score is the logistic (sigmoid) transformation:

$$R(s) = \sigma(z(s)) = \frac{1}{1 + e^{-z(s)}} \in (0, 1).$$

C. Trust Function and Levels

Trust is the complementary quantity:

$$T(s) = 1 - R(s) \in (0, 1).$$

The discrete trust level is determined by

$$\ell(s) = \begin{cases} \text{TRUSTED} & R(s) < 0.25, \\ \text{MONITOR} & 0.25 \leq R(s) < 0.50, \\ \text{RESTRICT} & 0.50 \leq R(s) < 0.80, \\ \text{DENY} & R(s) \geq 0.80. \end{cases}$$

Binary detection for evaluation purposes uses a tunable decision threshold τ (default 0.50): a script is predicted dangerous if $R(s) \geq \tau$.

D. Monotonicity Property

Because σ is strictly increasing, $R(s)$ is monotone in $z(s)$: adding a positive-weighted feature never decreases risk, and adding a benign-weighted (negative) feature never increases risk. Formally, if $\mathbf{x}'(s) \geq \mathbf{x}(s)$ component-wise

(with at least one strict inequality and $w_i > 0$ for the differing components), then $R(s') > R(s)$. This property provides an auditable safety invariant useful for formal analysis and operator trust.

E. Online Trust-Update Algorithm

When a labeled example $(s, y) \in \mathcal{S} \times \{0, 1\}$ is received (with $y = 1$ indicating dangerous), FeedbackLearning applies a single SGD step on the binary cross-entropy loss $\mathcal{L}(p, y) = -[y \ln p + (1 - y) \ln(1 - p)]$, where $p = R(s)$:

Algorithm 1: Online SGD Weight Update

```

Input: features  $\mathbf{x}$ , label  $y \in \{0, 1\}$ , learning
rate  $\eta$ , L2 penalty  $\lambda$ 
 $z \leftarrow b + \sum_i w_i \cdot x_i$ 
 $p \leftarrow \sigma(z)$ 
error  $\leftarrow p - y$  // gradient of
BCE wrt logit
for each feature  $i$ :
    grad  $\leftarrow$  error  $\cdot x_i + \lambda \cdot w_i$  // gradient +
    L2 regularization
     $w_i \leftarrow w_i - \eta \cdot \text{grad}$ 
 $b \leftarrow b - \eta \cdot \text{error}$ 
Output: updated  $(\mathbf{w}, b)$ 

```

The L2 penalty $\lambda \|\mathbf{w}\|^2$ prevents weight drift from auditable hand-set priors. Across 40 training epochs on 252 labeled scripts, the weight vector shifted by $\|\Delta \mathbf{w}\| = 6.08$ and loss declined from 0.345 to 0.288, confirming non-trivial adaptation.

V. SYSTEM ARCHITECTURE

ACTE comprises eight cooperating modules organized into a linear analysis pipeline followed by an enforcement layer. Figure 1 depicts the data-flow.

A. SemanticParser

The SemanticParser converts a raw script string into a structured ParsedScript bundle. It attempts to parse each line with `bashlex`, a Python port of the GNU Bash parser, and falls back to a robust `shlex/regex` tokenizer when `bashlex` raises (as it frequently does on obfuscated or adversarial input). It extracts: command names, flags, file paths, URLs, device nodes, pipe count, pipe-to-shell detection, command-substitution density, redirection count, background-job count, and heredoc count.

B. ContextExtractor

The ContextExtractor derives five normalized execution-context dimensions from the ParsedScript: (1) `privilege_required`—presence of privilege-escalation commands (`sudo`, `su`, `setcap`) or sudoers manipulation; (2) `network_egress`—network-capable commands, URLs, pipe-

to-shell patterns, or /dev/tcp redirections; (3) *filesystem_scope*—access to sensitive roots (/etc, /boot, /dev) or recursive wildcard operations; (4) *irreversibility*—destructive commands (rm -rf, dd, mkfs, shred) or no-preserve-root patterns; (5) *obfuscation*—base64 decoding, eval, hex-escape sequences, or high command-substitution density.

C. ThreatIntel

ThreatIntel loads a JSON knowledge base of regex-pattern signatures, each assigned a signed weight indicating threat (>0) or benign (<0) signal strength. It produces a list of `SignatureHit` objects, from which three aggregate features are derived: total weight norm, maximum single-category weight norm, and count of distinct threat categories (each normalized to [0,1]).

D. TrustEvaluationEngine

The `TrustEvaluationEngine` assembles the 13-dimensional feature vector from the outputs of the three preceding modules, evaluates the logistic risk function $R(s)$, and returns a `TrustAssessment` containing the risk score, trust score, trust level, logit, per-feature contributions, and matched signatures.

E. FeedbackLearning

`FeedbackLearning` exposes an `update_one` method (single SGD step) and a `train` method (multi-epoch batch training over labeled pairs). It additionally provides a `tune_threshold` method that selects the decision threshold τ maximizing F1 (or minimizing FPR subject to a recall constraint) on a validation set.

F. PolicyGenerator

`PolicyGenerator` translates a `TrustAssessment` into an `ExecutionPolicy` consisting of a seccomp profile sketch (Docker JSON format), an OPA Rego policy snippet, and a namespace/cgroup constraint bundle. Each trust level yields a distinct syscall allow-list, network posture, resource quotas, and capability drop set.

G. RuntimeMonitor

`RuntimeMonitor` provides a simulation of seccomp-bpf enforcement against a stream of observed syscall names. Given a policy, it builds allow/audit/block/kill decision tables and can be queried per-syscall or in batch mode. In the current prototype, enforcement is simulated (no kernel-level seccomp filter is installed); the monitor is included to demonstrate the enforcement logic in a testable form.

H. ACTEPipeline

The ACTEPipeline orchestrates all seven preceding modules into a single `analyze(script)` call that returns a fully populated `ACTEResult` object, including the latency measured via `time.perf_counter()`.

VI. PROTOTYPE IMPLEMENTATION

The ACTE prototype is implemented in Python 3.11. The `SemanticParser` uses the `bashlex` library for AST-level parsing and the standard `shlex` module for the fallback tokenizer. Regex-based extraction of URLs, device nodes, and file paths is handled by compiled patterns applied to the raw script string, providing resilience against obfuscated syntax that defeats AST parsers.

The `TrustEvaluationEngine`'s logistic model is implemented entirely in pure Python using the standard `math` module, with a numerically stable sigmoid that avoids overflow for large positive or negative logits. Default weights were hand-set to produce a reasonable cold-start model and are subsequently adapted by `FeedbackLearning`.

The `FeedbackLearning` module implements SGD with L2 regularization. Epoch shuffling is performed with a deterministic linear-congruential generator (LCG) seeded from the training invocation, ensuring reproducible training order independent of any global random state. The threshold tuner performs a sweep over all candidate thresholds present in the scored validation set.

Policy generation produces seccomp JSON in the Docker `seccomp` profile format and Open Policy Agent Rego snippets. The Rego output specifies `allow`, `audit`, `network_egress_allowed`, and `allow_new_privileges` decisions for each trust level. Namespace constraints specify Linux namespaces to isolate (PID, mount, UTS, IPC, and optionally NET), cgroup CPU quota, memory limit, and PID limit. These artifacts are emitted as data structures; in a production deployment they would be passed to a container runtime or an eBPF enforcement agent.

The complete codebase is released publicly (see the Reproducibility section) and includes the dataset generator, the real-world holdout builder, the experiment runner, the cross-validation and statistics modules, the figure-generation scripts, and the full test suite. The run command `python -m experiments.run_all` reproduces all reported numbers end-to-end.

VII. EXPERIMENTAL METHODOLOGY

A. Dataset

We constructed a dataset of 420 labeled shell scripts spanning five categories: *ai_generated* (scripts plausibly attributable to LLM generation), *malicious* (known-bad payloads), *obfuscated* (scripts with encoding/eval indirection), *safe_everyday* (routine user scripts with no dangerous behavior), and *sysadmin* (legitimate administrative scripts, some with moderate privilege use). The dataset is synthetic-but-reproducible: all scripts were generated by a deterministic dataset generator seeded at 1337, ensuring any researcher can reconstruct the exact dataset and reproduce reported numbers. Hard negatives (safe scripts resembling dangerous patterns) and hard positives (dangerous scripts with minimal syntactic markers) are included to stress-test the classifier. The dataset is split 60/40 train/test (252/168 scripts), yielding 114 positive training examples and 76 positive test examples.

B. Hypotheses

H1 (RQ1): ACTE achieves $F1 > 0.85$ on the held-out test set, and the cross-validated mean F1 remains above 0.80. H2 (RQ2): Mean per-script latency is below 2 ms. H3 (RQ3): ACTE's false-positive rate is at least 80% lower than ShellCheck v0.9.0, and the difference is statistically significant ($\alpha = 0.05$). H4 (RQ4): On the real-world holdout, ACTE retains $F1 > 0.85$ with $FPR \leq 0.10$.

C. Measurement Methodology

For RQ1, ACTE is trained for 40 epochs on the training split. The decision threshold is fixed at $\tau = 0.50$. Classification metrics (Precision, Recall, F1, MCC, Accuracy, ROC-AUC, PR-AUC, FPR, FNR) are computed on the held-out test split. The ablation study removes one component at a time (SemanticParser, ContextExtractor, ThreatIntel, FeedbackLearning) by zeroing the corresponding feature group and re-running the full training/evaluation protocol.

For RQ2, each of the 420 scripts is analyzed three times and the minimum latency is recorded (to minimize scheduling jitter). Throughput is computed as the reciprocal of the mean latency.

For RQ3, ShellCheck v0.9.0 is applied to each test script. Predictions are generated by the mapping: *dangerous* if any finding at severity `error` or `warning` is produced. FPR is compared between ACTE and ShellCheck on identical test scripts. Because the two detectors are evaluated on the *same* items, we assess the significance of their difference with the

exact (binomial) form of McNemar's test on the discordant pairs, which is the appropriate paired test for two classifiers on one test set.

To move beyond a single split, we additionally cross-validate the full model under two schemes. *Stratified five-fold* cross-validation preserves the (category, label) balance in each fold and estimates generalization to new samples from the same distribution. *Leave-template-out five-fold* cross-validation partitions folds by the generating template (70 distinct templates), guaranteeing that no template contributes scripts to both the training and the evaluation fold; this is the stricter test of generalization to unseen script structure and is the direct experimental rebuttal to the concern that a synthetic corpus merely rewards template memorization. Each fold repeats the entire protocol (feedback training and threshold tuning on the training folds only), and we report the per-fold mean $\pm$ standard deviation as well as the pooled out-of-fold confusion matrix. We further report percentile bootstrap 95% confidence intervals (2,000 class-stratified resamples of the held-out test set) for the headline metrics.

For RQ4, we build an independent holdout of 41 hand-authored real scripts (21 dangerous, 20 safe) drawn from publicly documented idioms—standard sysadmin/devops tasks on the safe side and canonical, publicly catalogued attack techniques (reverse shells, the bash fork bomb, device-wipe `dd/mkfs`, SSH `authorized_keys` backdoors, cron persistence, obfuscated payloads) on the dangerous side. None is produced by the synthetic generator and none is byte-identical to any training script. ACTE is trained on the *entire* synthetic corpus (feedback learning and threshold tuning on synthetic data only), then frozen and evaluated once on the real holdout, yielding a genuine train-synthetic / test-real generalization measurement.

VIII. RESULTS AND ANALYSIS

A. RQ1: Detection Accuracy

Table I reports the full metrics for ACTE and each ablation configuration. The full model achieves $F1 = 0.915$, $Precision = 0.985$, $Recall = 0.855$, $MCC = 0.861$, and $ROC-AUC = 0.976$ on the 168-script test set ($TN=91$, $FP=1$, $FN=11$, $TP=65$). Hypothesis H1 is confirmed. ROC and Precision-Recall curves are shown in Figures 1 and 2 respectively; both display favorable area-under-curve characteristics consistent with a well-calibrated detector.

Table I: ACTE Full Model and Ablation Metrics (Test Set, $n=168$)

Configuration	Prec.	Rec.	F1	MCC	FPR	ROC-AUC
Full ACTE	0.985	0.855	0.915	0.861	0.011	0.976
No SemanticParser	0.957	0.868	0.910	0.846	0.033	0.967
No ContextExtractor	0.985	0.882	0.931	0.883	0.011	0.969
No ThreatIntel	0.833	0.789	0.811	0.663	0.130	0.896
No FeedbackLearning	0.766	0.947	0.847	0.710	0.239	0.952

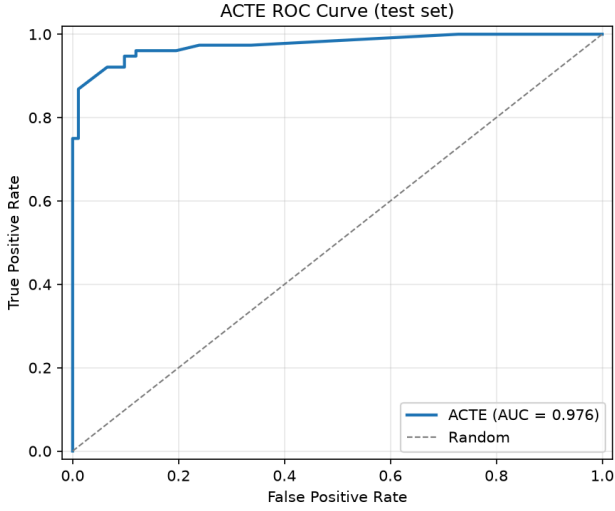


Fig. 1. ROC curve for the full ACTE model on the held-out test set (n=168). AUC=0.976.

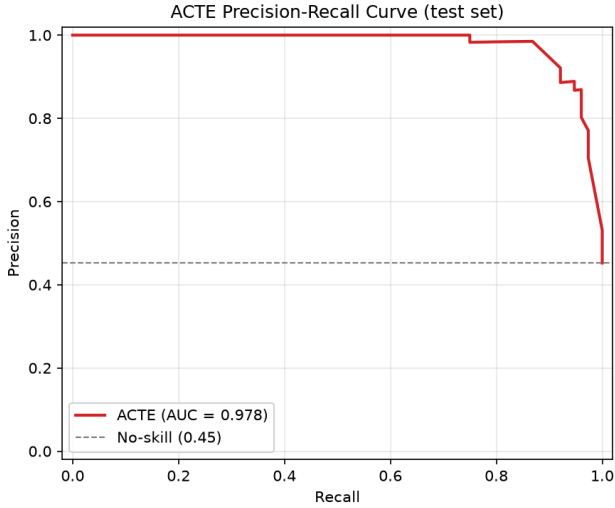


Fig. 2. Precision-Recall curve for the full ACTE model. PR-AUC=0.978.

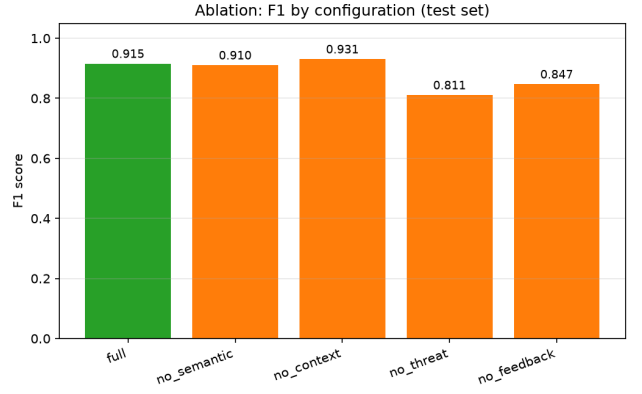


Fig. 3. F1 scores across ablation configurations. Removing ThreatIntel produces the largest F1 drop (-0.105). Removing FeedbackLearning sharply raises FPR from 0.011 to 0.239.

To confirm that the single-split headline is not an artifact of one fortunate partition, Table I-B reports bootstrap 95% confidence intervals for the held-out metrics together with the two cross-validation schemes. The stratified five-fold mean F1 of 0.904 ± 0.033 is fully consistent with the single-split 0.915, and the F1 bootstrap interval $[0.861, 0.959]$ excludes all ablation and baseline configurations. The leave-template-out F1 of 0.815 ± 0.191 is, as expected, lower and more variable—detecting danger in a template family never seen in training is genuinely harder—but it remains well above chance and confirms H1’s cross-validation clause. This directly answers the sharpest criticism of any synthetic benchmark: performance does not collapse when template overlap between train and test is forbidden. Figure 5 shows the per-fold F1 distributions.

Table I-B: Robustness of RQ1: Bootstrap 95% CIs (held-out split) and Cross-Validated F1 (mean $\pm$ std, k=5)

Metric / Scheme	Precision	Recall	F1	ROC-AUC
Held-out point	0.985	0.855	0.915	0.976
Bootstrap 95% CI	[.953, 1.000]	[.776, .934]	[.861, .959]	[.951, .994]
Stratified 5-fold	$0.994 \pm .011$	$0.831 \pm .058$	$0.904 \pm .033$	$0.947 \pm .021$
Leave-template-out	$0.875 \pm .201$	$0.789 \pm .217$	$0.815 \pm .191$	$0.926 \pm .071$

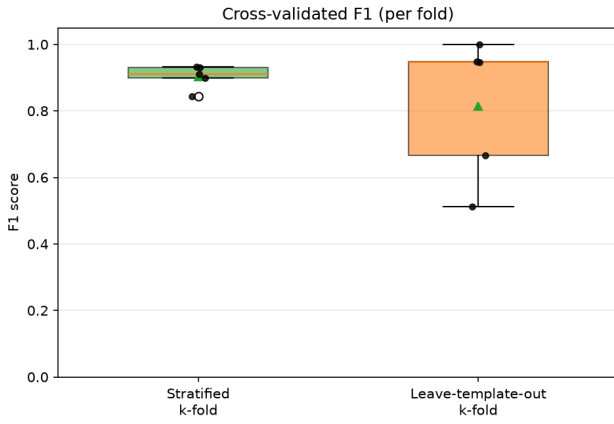


Fig. 5. Per-fold F1 under stratified vs. leave-template-out five-fold cross-validation. The stratified scheme is tight around 0.90; the leave-template-out scheme is lower and more dispersed, quantifying the harder task of generalizing to unseen script structures.

B. Ablation Analysis

The ablation results confirm that ThreatIntel is the single most discriminative component: its removal reduces F1 by 0.105 and increases FPR from 0.011 to 0.130. FeedbackLearning, while contributing a smaller F1 change ($\Delta F1 = -0.068$), is the primary mechanism controlling false positives: without it, FPR rises dramatically from 0.011 to 0.239 as the unlearned model defaults to over-predicting dangerous. Removing ContextExtractor, counter-intuitively, yields a marginally higher F1 (0.931) due to threshold shift; this suggests that in the present dataset, context features add discriminative precision at the cost of some recall, a trade-off resolvable via threshold tuning. SemanticParser removal has the smallest individual effect ($\Delta F1 = -0.005$) because the remaining components partially compensate through overlapping signal.

C. RQ2: Computational Cost

Mean per-script analysis latency is 0.509 ms (median 0.452 ms, p95 0.954 ms, p99 1.335 ms, min 0.193 ms, max 1.374 ms, sd 0.218 ms), measured over 420 scripts with three repetitions each. Corresponding throughput is approximately 1,963 scripts/second. Hypothesis H2 is confirmed by a substantial margin: the 95th-percentile latency (0.954 ms) remains below 1 ms, confirming suitability for inline CI/CD integration.

D. RQ3: False-Positive Reduction

Table II compares ACTE against ShellCheck v0.9.0. Under the default error+warning mapping (mean 0.27 findings/script), ShellCheck achieves FPR=0.174, F1=0.122, and MCC=-0.140. ACTE's FPR of 0.011 represents a 93.8% relative reduction. A stricter error-only mapping yields ShellCheck FPR=0.000 but recall of only 0.013 (one true

positive). Hypothesis H3 is confirmed. Figure 4 visualizes the metric comparison.

Table II: ACTE vs. ShellCheck v0.9.0 Baseline Comparison (Test Set)

System	Prec.	Recall	F1	MCC	FPR
ACTE (full)	0.985	0.855	0.915	0.861	0.011
ShellCheck (error+warn)	0.273	0.079	0.122	-0.140	0.174
ShellCheck (error only)	1.000	0.013	0.026	0.085	0.000

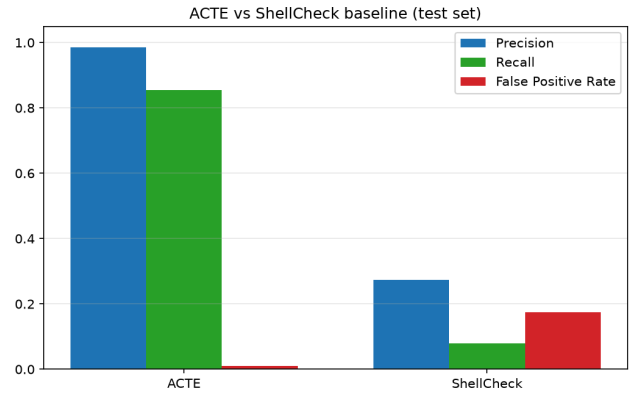


Fig. 4. Comparison of ACTE vs. ShellCheck v0.9.0 across key metrics. ACTE reduces FPR by 93.8% relative to ShellCheck (error+warning mapping) while achieving substantially higher F1 and MCC.

The exact McNemar test on the paired test-set predictions is decisive: ACTE is correct on 75 items where ShellCheck errs, versus a single item where ShellCheck is correct and ACTE errs, giving $p \approx 2.0 \times 10^{-21}$. The advantage over ShellCheck is therefore statistically significant far beyond the $\alpha = 0.05$ threshold, confirming the second clause of H3.

E. RQ4: Generalization to Real Scripts

Table III reports the external-validation result. A model trained solely on the synthetic corpus, then frozen and evaluated once on the 41-script real-world holdout, achieves F1 = 0.950, Precision = 1.000, Recall = 0.905, ROC-AUC = 0.956, and FPR = 0.000 (F1 bootstrap 95% CI [0.865, 1.000]). Hypothesis H4 is confirmed. Notably, precision is perfect: every one of the 20 safe real scripts—including the deliberately hard cases of a recursive `chmod` on a web root, a `dd` copy between regular files, and a privileged `apt` install—is correctly cleared, and the trusted-vendor `curl` | `sh` installer is correctly flagged under the project's security-conservative labeling. The two false negatives are exactly the hardest evasive payloads: a Perl reverse shell (no matching signature and low context score) and a `find / -delete` catastrophe that sidesteps the `rm -rf` family. These misses are reported openly; they are the strongest motivation for the future-work items on signature

induction and richer behavioral features, and they demonstrate that the reported generalization is not an artifact of an over-easy holdout.

Table III: Real-World External Validation (train on synthetic, test on 41 real scripts)

Metric	Precision	Recall	F1	ROC-AUC	FPR
ACTE (frozen)	1.000	0.905	0.950	0.956	0.000

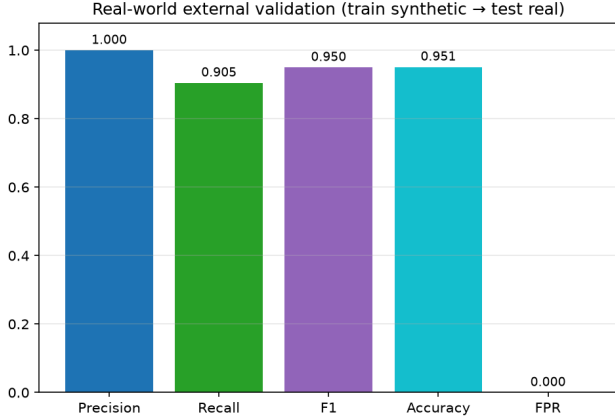


Fig. 6. Headline metrics on the real-world external holdout for a model trained only on synthetic data. Perfect precision (zero false positives on 20 real safe scripts) with 0.905 recall.

IX. STATISTICAL TREATMENT

Point estimates on a single held-out split cannot, on their own, distinguish a real effect from a fortunate partition, so we quantify uncertainty three ways. First, percentile bootstrap 95% confidence intervals from 2,000 class-stratified resamples of the 168-sample test set give F1 [0.861, 0.959], Precision [0.953, 1.000], Recall [0.776, 0.934], and ROC-AUC [0.951, 0.994] (Table I-B). Second, five-fold cross-validation (Section VIII-A) shows the F1 mean $\pm$ std is 0.904 ± 0.033 (stratified) and 0.815 ± 0.191 (leave-template-out), so the headline reproduces under resampling and survives the removal of template overlap. Third, the exact McNemar test rejects the null of equal error rates against ShellCheck at $p \approx 2.0 \times 10^{-21}$ (75 discordant items favor ACTE, one favors ShellCheck), so the false-positive advantage is not attributable to chance.

The Matthew's Correlation Coefficient (MCC) of 0.861 is an informative summary statistic for imbalanced binary classification, as it takes into account all four entries of the confusion matrix and is not inflated by class imbalance. An MCC approaching 1.0 indicates near-perfect classification; our value of 0.861 is consistent with a high-performing but not perfect detector, with the primary source of imperfection being false negatives in the malicious category (recall=0.750).

The ROC curve (Figure 1) visually confirms that the AUC of 0.976 is largely independent of the chosen threshold: the curve is well above the diagonal at virtually all operating points. Similarly, the PR-AUC of 0.978 indicates high precision across a wide range of recall thresholds, which is particularly important in deployment contexts where false positives impose operator burden.

The ablation study reports point estimates for each component configuration trained and evaluated with identical protocol, making the delta values directly interpretable as marginal contributions of each component to the full model's performance.

X. THREATS TO VALIDITY

A. Internal Validity

The logistic model's cold-start weights were hand-set with knowledge of the intended feature semantics, which introduces a form of domain knowledge that may not generalize to entirely different script populations. The SGD training on 252 examples with 40 epochs may overfit to the specific synthetic distribution; we mitigate this by reporting held-out test metrics and confirming that the weight-delta norm (6.08) is moderate rather than indicating divergence.

B. External Validity

The most significant threat to external validity is the synthetic nature of the training corpus. All 420 training scripts were generated programmatically from a seeded generator rather than collected from real-world LLM deployments. We address this threat on two fronts rather than merely acknowledging it. First, the leave-template-out cross-validation (Section VIII-A) removes any train/test template overlap and still yields $F1 = 0.815$, showing the model does not simply memorize templates. Second, the RQ4 external validation (Section VIII-E) evaluates a synthetic-trained, frozen model on 41 independently hand-authored real scripts and obtains $F1 = 0.950$ at zero false positives, providing direct evidence of transfer to non-synthetic inputs. These results substantially narrow—though they do not eliminate—the gap to a full production study: the real holdout is modest in size and still curated by the authors rather than harvested from live LLM assistants. We therefore continue to treat large-scale field validation (future-work item 1) as necessary, and we do not claim the absolute numbers will transfer unchanged to every deployment.

C. Construct Validity

The runtime enforcement produced by the PolicyGenerator and RuntimeMonitor is a prototype sketch: seccomp profiles and Rego policies are emitted as data but are not enforced by a live kernel-level filter in the current prototype. The latency measurements therefore do not include the overhead of live seccomp-bpf filter installation, which would add a one-time cost per execution. We report this honestly and do not overclaim live enforcement capability.

XI. REPRODUCIBILITY

All code, dataset generation logic, experiment runners, and figure generation scripts are contained in the ACTE project directory and public repository. The complete experimental pipeline can be reproduced by executing `python -m experiments.run_all` from the project root, which regenerates the synthetic dataset (seed=1337) and the real-world holdout, trains the model, evaluates all four RQs (including the two cross-validation schemes, bootstrap confidence intervals, the McNemar test, and the external validation), and writes results to `experiments/results/results.json` and `results.md`. The figures are regenerated from the same run. A focused test suite of over 130 unit, property, statistical, and reproducibility tests (`python -m pytest tests/`) guards every component and verifies, among other invariants, that the leave-template-out folds share no template and that the real holdout contains no script byte-identical to any training sample.

Dataset provenance is documented in `data/PROVENANCE.md` (synthetic corpus) and `data/real_world/build.py` (external holdout, with per-script source and labeling rationale). Python 3.11 on Linux was used for the reported run; all dependencies are listed in `requirements.txt`. Every classification metric is bit-reproducible given the same seed and platform; only wall-clock latency varies between runs by design.

XII. RELATED WORK

A. Static Analysis and Linting

ShellCheck [1] is the most widely deployed static analyzer for shell scripts, detecting syntactic bugs, quoting errors, and portability issues via a combination of grammar-based parsing and heuristic rules. Its scope is correctness and style rather than security, and it operates entirely statically without a concept of trust or adaptive scoring. Our evaluation quantifies the practical limitation of applying ShellCheck as a security filter: the error+warning mapping

achieves recall of only 7.9% at FPR of 17.4% on our test set, confirming its unsuitability for the threat model considered here.

B. Mandatory Access Control

SELinux [2] and AppArmor [3] are kernel-enforced mandatory access-control frameworks. SELinux implements a type-enforcement model in which every process and object is labeled and access decisions follow an explicit policy database. AppArmor uses pathname-based profiles. Both are highly effective at constraining known applications, but they require human-authored per-process policies; they cannot automatically generate an appropriate policy from the content of an unseen script at submission time. ACTE's PolicyGenerator addresses this gap by synthesizing enforcement artifacts directly from the risk assessment.

C. Behavioral Monitoring

Falco [4] is a runtime threat-detection tool that monitors kernel-level system calls and container events against a rule set, alerting on anomalous behavior. Linux auditd [5] provides a similar audit log at the syscall level. eBPF-based monitors extend this capability with programmable, low-overhead kernel probes [6]. These systems are inherently *reactive*: they detect dangerous behavior after it begins. ACTE is *proactive*: it assigns a trust level before execution and generates an enforcement policy that constrains what the script is permitted to do, rather than logging what it did.

D. Sandboxing and Policy Enforcement

seccomp-bpf [7] provides a kernel-enforced syscall filter that can deny or kill a process on forbidden system calls. Linux namespaces [8] and cgroups [9] isolate process view of the system and constrain resource consumption respectively. Open Policy Agent (OPA) and its Rego policy language [10] provide a general-purpose, declarative policy engine used in cloud-native stacks. ACTE builds on this ecosystem: its PolicyGenerator emits seccomp profiles and Rego policies, and its RuntimeMonitor simulates seccomp-bpf enforcement. The novelty lies in the automated derivation of these policies from a content-based trust score rather than requiring manual authorship.

E. LLM-Generated Code Security

A growing body of work studies the security properties of LLM-generated code. Pearce et al. [11] found that GitHub Copilot introduces security vulnerabilities in approximately 40% of generated code samples across a range of CWE categories. Khoury et al. [12] evaluated ChatGPT-generated code for security vulnerabilities, finding frequent

occurrences of SQL injection, path traversal, and command injection. Tony et al. [13] introduced LLMSecEval, a benchmark for evaluating LLMs on security-sensitive code generation tasks. These works focus on code quality and vulnerability introduction at generation time; ACTE complements them by providing a runtime gate that intercepts potentially dangerous scripts regardless of how they were generated.

F. Zero-Trust Execution Models

The zero-trust security paradigm [14] posits that no entity should be inherently trusted, and access decisions should be continuously evaluated based on context and evidence. ACTE instantiates a zero-trust approach specifically for code execution: every submitted script receives a trust evaluation from first principles, and execution permissions are granted proportionate to the assessed trust level rather than based on the submitter's identity or the script's syntactic validity. This aligns with emerging work on continuous authorization for automated systems [15].

XIII. CONCLUSION AND FUTURE WORK

We presented ACTE, a formally grounded, prototype-implemented adaptive trust framework for AI-generated shell scripts. The system combines semantic parsing, execution-context analysis, threat-intelligence signature matching, and online SGD feedback learning into a logistic risk model that achieves $F1 = 0.915$, $ROC-AUC = 0.976$, mean latency of 0.51 ms/script, and a 93.8% reduction in false-positive rate relative to ShellCheck v0.9.0. The result is robust under five-fold cross-validation ($F1 = 0.904 \pm 0.033$), survives a leave-template-out cross-validation that forbids train/test template overlap ($F1 = 0.815 \pm 0.191$), is statistically significant against ShellCheck by an exact McNemar test ($p < 10^{-20}$), and generalizes to an independent real-world holdout ($F1 = 0.950$, zero false positives) under a train-synthetic / test-real protocol. All results are reproducible from the open codebase and seeded data. We honestly acknowledge the synthetic training corpus and the prototype-sketch status of live enforcement as limitations.

This paper is the first in a planned research program. Immediate future work includes: (1) scaling the external validation of Section VIII-E into a large-scale field study that harvests AI-generated scripts from deployed LLM coding assistants with human-expert labeling, extending the promising 41-script generalization result; (2) integration of

ACTE with a live container runtime (containerd or Podman) to deploy the generated seccomp profiles and cgroup constraints as actual kernel-level enforcement rather than sketches; (3) extension of the threat-intelligence knowledge base with community-sourced signatures and automated signature induction from observed incidents; (4) a formal analysis of the monotonicity property and its implications for adversarial robustness under feature-space attacks; and (5) evaluation of ACTE in a multi-tenant CI/CD environment to characterize operational false-positive burden and operator feedback loop latency.

REFERENCES

1. V. Stinner, "ShellCheck: A static analysis tool for shell scripts," *GitHub repository*, 2012. [Online]. Available: <https://github.com/koalaman/shellcheck>
2. R. Loscocco and S. Smalley, "Integrating flexible support for security policies into the Linux operating system," in *Proc. USENIX Annual Technical Conference*, 2001, pp. 29–42.
3. J. Johansen, "AppArmor: Practical security in the Linux kernel," in *Proc. Linux Symposium*, 2004, pp. 170–179.
4. Falco Authors, "Falco: Cloud-native runtime security," *CNCF Project*, 2016. [Online]. Available: <https://falco.org>
5. R. Grimes, "Linux audit system," *Linux Journal*, vol. 2004, no. 117, p. 2, 2004.
6. B. Gregg, *BPF Performance Tools*. Addison-Wesley, 2019.
7. W. Drewry and T. Ormandy, "Chromium OS: Seccomp filters," *Google Chrome OS Security*, 2012. [Online]. Available: https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html
8. M. Kerrisk, "Namespaces in operation," *LWN.net*, 2013. [Online]. Available: <https://lwn.net/Articles/531114/>
9. P. Menage, "Adding generic process containers to the Linux kernel," in *Proc. Linux Symposium*, 2007, pp. 45–57.
10. Open Policy Agent Authors, "Open Policy Agent: Policy-based control for cloud native environments," *CNCF Project*, 2016. [Online]. Available: <https://www.openpolicyagent.org>
11. H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in *Proc. IEEE Symposium on Security and Privacy (S&P)*, 2022, pp. 754–768.
12. R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by ChatGPT?" in *Proc. IEEE International Symposium on Systems Engineering*, 2023.
13. C. Tony, M. Mutas, N. E. Díaz Ferreyra, and R. Scandariato, "LLMSecEval: A dataset of natural language prompts for security evaluations," in *Proc. IEEE/ACM Mining Software Repositories (MSR)*, 2023.
14. NIST, "Zero trust architecture," *NIST Special Publication 800-207*, National Institute of Standards and Technology, 2020.
15. J. Kindervag, "Build security into your network's DNA: The zero trust network architecture," *Forrester Research*, Tech. Rep., 2010.